

Día D — sábado, instrucciones en caliente

La línea base ya está construida, probada y desplegada. Esta hoja cubre solo lo que pasa **en vivo**, cuando el profesor dicte las dos Historias de Usuario nuevas. Los comandos se copian tal cual; lo **resaltado** es lo único que se cambia.

Antes de que empiece la clase — los cinco

Cada uno, en su máquina, dentro de la carpeta `speckit-ai-generator`:

```
git checkout main
git pull origin main
npm install
npm test
```

Tiene que decir `72 passed` o más y `0 failed`. Si sale rojo, avisa al grupo **ahora**. Deja tu agente abierto en esa carpeta.

- **Tomás** — confirma que su `.env.local` con las ocho llaves sigue en su máquina.
- **Santiago** — abre `https://_____vercel.app` en el celular y comprueba que carga.
- **Todos** — hotspot del celular encendido, por si falla el wifi del salón.

La secuencia en vivo — 10 minutos

1 Sergio escribe la especificación

≈ 3 min · rama `main`

Proyecta la pantalla. Primero deja la rama limpia, después pega los dos prompts en el agente, uno tras otro:

```
git checkout main
git pull origin main
```

```
/speckit-specify Agrega dos User Stories nuevas al final de specs/001-ai-media-generator/spec.md.
HU 1: "historia literal que dictó el profesor"
HU 2: "segunda historia literal"
NO crees una feature nueva. NO corras create-new-feature.sh. NO cambies de rama.
Solo edita ese spec.md que ya existe.
```

```
/speckit-tasks Agrega al final de specs/001-ai-media-generator/tasks.md SOLO los tickets de
las dos User Stories que acabo de agregar. Numeralos T040, T041 y T042.
Exactamente tres tickets, ni uno mas.
Cada ticket con su lista Owns, y las tres listas sin ningun archivo en comun entre ellas.
NO regeneres ni reenumeres los 39 tickets que ya estan.
```

Sube spec y tickets para que los demás los bajen:

```
git add -A
git commit -m "HU en vivo: spec y tickets T040-T042"
git push origin main
```

Antes de seguir, di en voz alta quién toma cada ticket: **Mateo** → **T040**, **Johan** → **T041**, **Tomás** → **T042**. Si salieron más de tres tickets, **Santiago** toma el cuarto con el mismo bloque del paso 2.

2 Mateo, Johan y Tomás construyen, los tres a la vez

≈ 5 min

Cada uno copia **su** columna. Son idénticas salvo el número del ticket.

MATEO

```
git checkout main
git pull origin main
git checkout -b hu-viva-t040
# en el agente:
/speckit-implement T040
npm test
git add -A
git commit -m "T040 HU en vivo"
git push -u origin hu-viva-t040
```

JOHAN

```
git checkout main
git pull origin main
git checkout -b hu-viva-t041
# en el agente:
/speckit-implement T041
npm test
git add -A
git commit -m "T041 HU en vivo"
git push -u origin hu-viva-t041
```

TOMÁS

```
git checkout main
git pull origin main
git checkout -b hu-viva-t042
# en el agente:
/speckit-implement T042
npm test
git add -A
git commit -m "T042 HU en vivo"
git push -u origin hu-viva-t042
```

Si npm test sale rojo, no hagas push. Dilo en voz alta y pídele al agente: «*la prueba X está fallando, arrégla y vuelve a correr npm test*». Que el método atrape el error delante del profesor juega a favor. Cuando tu `git push` termine, anúncialo: «*T040 arriba*» — **Santiago** no arranca hasta oír los tres.

3 Santiago junta las ramas y despliega

≈ 2 min · rama `main`

Solo cuando los tres hayan dicho que subieron:

```
git checkout main
git pull origin main
git fetch origin
```

Un merge por línea. Espera a que termine cada uno antes de escribir el siguiente:

```
git merge origin/hu-viva-t040
git merge origin/hu-viva-t041
git merge origin/hu-viva-t042
```

Con los tres adentro, prueba y sube:

```
npm test
git push origin main
```

Ese push a `main` dispara el despliegue en Vercel por sí solo. Nadie corre un comando de deploy.

4 Todos muestran la app funcionando

≈ 1 min

1. **Santiago** proyecta el panel de Vercel y espera el check verde del deploy (1–2 min).
2. Con el deploy en verde, abre <https://.vercel.app> en el celular.
3. Recorre la Historia nueva de punta a punta, en el celular, delante del profesor.

Si el deploy todavía no termina, **Santiago** corre `npm run dev` y muestra la app en `http://localhost:3000` mientras tanto: es su máquina la que tiene las llaves reales.

Si algo se rompe

Qué pasó	Quién lo arregla	Qué hace, exactamente
Conflicto en el merge	Santiago	En <code>package.json</code> : dejar las dependencias de los dos lados. En cualquier <code>lib/*.js</code> : dejar la implementación real y borrar el stub. Después <code>git add -A</code> y <code>git commit</code> .
El merge se enredó	Santiago	<code>git merge --abort</code> y volver a intentar solo con esa rama. No pierde nada de lo ya mergeado.
El agente toca archivos de otro	El dueño del ticket	Pararlo y decirle: « <i>revierte eso, solo puedes tocar los archivos de la lista Owns de mi ticket</i> ».
El agente se pierde	Sergio	<code>/speckit-analyze</code> — no cambia nada, muestra qué se contradice entre spec, plan y tickets.
<code>npm test</code> en rojo	Quien lo corrió	No hacer push. Decirle al agente qué prueba falla y que la arregle. Volver a correr <code>npm test</code> .
Vercel lento o caído	Santiago	<code>npm run dev</code> y demo en <code>http://localhost:3000</code> .
Se cayó el wifi	Todos	Hotspot del celular. La app corre en local sin llaves ni internet gracias a los mocks del T008.

Las cuatro reglas que no se rompen

1. Es `/speckit-implement`, **no** `/implement`.
2. Nunca `/speckit-implement` a secas: sin el número del ticket construye los 42, los tuyos y los de todos.
3. Los merges van **uno por uno**. `git merge A B C` se cancela entero al primer conflicto.
4. Nadie hace `git push origin main` en vivo salvo **Sergio** en el paso 1 y **Santiago** en el paso 3. Los demás empujan solo a su rama `hu-viva-*`.